

# LeadScrapper

---

*Complete Architecture Documentation — Google Maps Lead Generation & Management Platform*

**Project:** leadscraper.pixnom.com

**Date:** May 2026

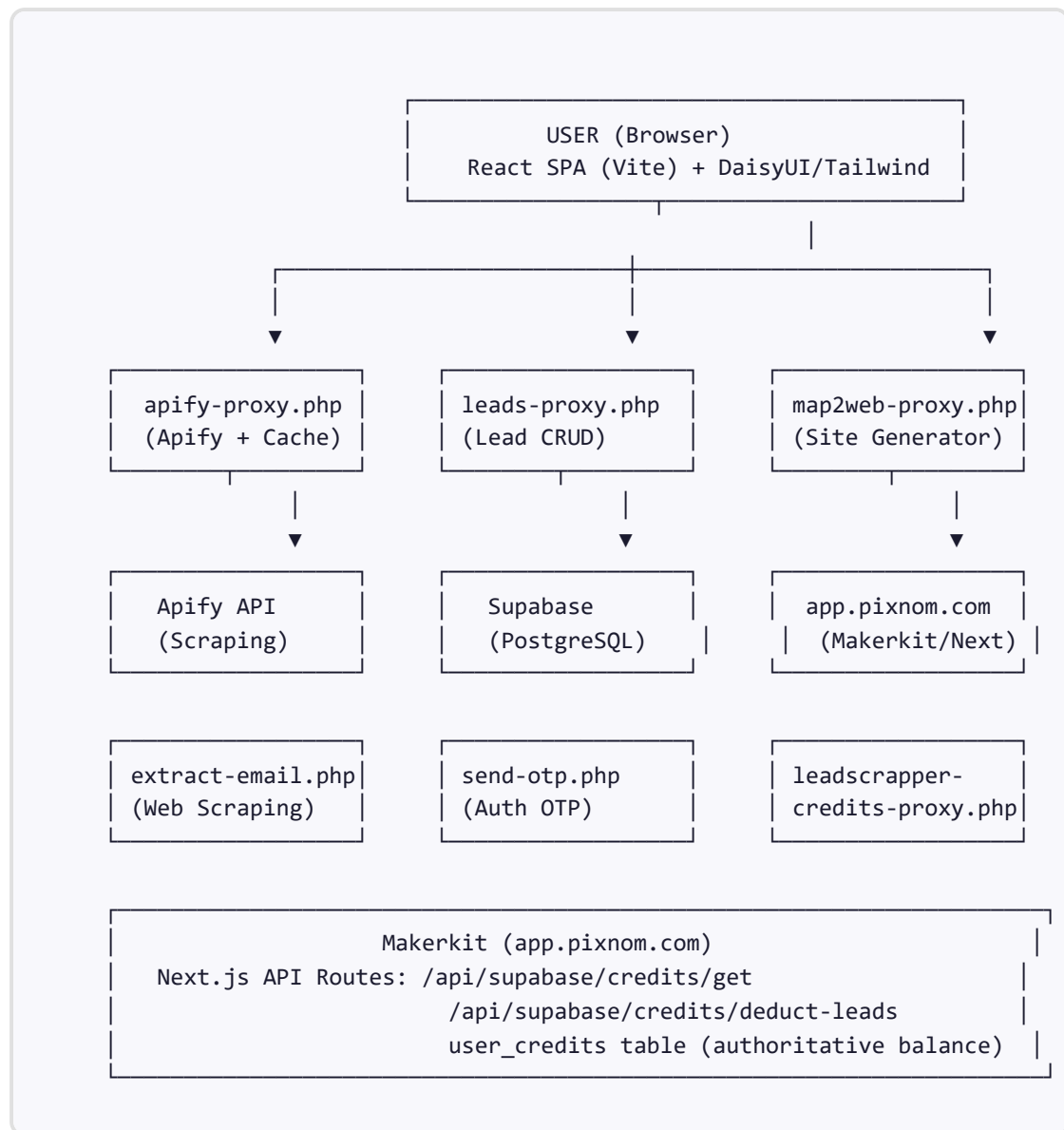
**Stack:** React (Vite) + PHP Backend + Supabase + Apify + Makerkit (Next.js)

## Table of Contents

1. System Overview & High-Level Architecture
2. The Complete Lead Search Flow (10 Steps)
3. Frontend Architecture (React + Vite)
4. PHP Backend Layer
5. Apify Integration & Key Rotation
6. Credit & Billing System
7. Supabase Data Layer
8. Makerkit Integration (Next.js)
9. Email Extraction System
10. Map2Web Site Generation Integration
11. Authentication & Security
12. All PHP Proxy Files Reference
13. Database Schema & Migrations
14. Architecture Observations & Gotchas

# 1. System Overview & High-Level Architecture

LeadScrapper is a lead generation platform that scrapes Google Maps for business listings using Apify, scores them by lead quality, manages them in a CRM-like interface, extracts emails, generates outreach emails, and can create full websites for leads via Map2Web integration.



## Tech Stack Summary

| Layer    | Technology                           | Purpose              |
|----------|--------------------------------------|----------------------|
| Frontend | React 19 + Vite + DaisyUI + Tailwind | SPA with CRM-like UI |

|          |                                       |                                            |
|----------|---------------------------------------|--------------------------------------------|
| Backend  | PHP (cURL-based proxies)              | Server-side API proxying, caching, billing |
| Scraping | Apify (compass~crawler-google-places) | Google Maps data extraction                |
| Database | Supabase (PostgreSQL)                 | Leads, credits, cache, audit logs          |
| Billing  | Makerkit (Next.js on app.pixnom.com)  | Authoritative credit balance & deductions  |
| Email    | Resend API                            | OTP delivery for auth                      |
| Anti-Bot | Cloudflare Turnstile                  | CAPTCHA gate before login                  |
| Site Gen | Map2Web (app.pixnom.com)              | Generate websites for leads                |
| Hosting  | Pixnom server (PHP)                   | leadscraper.pixnom.com                     |

### Key Insight

The PHP layer is NOT a traditional MVC framework. Each PHP file is a standalone API proxy endpoint. There's no router, no framework — just raw PHP files with cURL calls. The React frontend talks directly to these PHP endpoints.

## 2. The Complete Lead Search Flow (10 Steps)

### 1 User Input

User enters a keyword (e.g., "plumber"), selects country/state/city or ZIP code, and clicks Search. The `LeadSearch` component collects: keyword, locationQuery (or postalCode + countryCode), maxCrawledPlacesPerSearch.

**File:** `components/LeadSearch.jsx`

### 2 Credit Pre-Check

Before calling Apify, the proxy checks the user's credit balance via `credits_get_balance()`. This calls Makerkit's `/api/supabase/credits/get` endpoint. If balance < 0.01 credit, returns 402 (Insufficient Credits).

**File:** `apify-proxy.php` (lines 428-455)

### 3 Supabase Cache Check

Builds a cache key from `keyword|location` (e.g., `"plumber|zip:90001|us"`). Queries the `leadscraper_leads_data` table for existing rows with `UserEmail='__cache__'`. If cache has any places for this key, **NEVER re-scrapes** — serves from cache directly with a synthetic runId.

**File:** `apify-proxy.php` (lines 457-508), `lib/supabase_cache.php`

### 4 Apify Scraping (on cache miss)

On cache miss, calls the Apify `compass~crawler-google-places` actor. The proxy rotates through up to 21 API keys (`APIFY_KEY_1` through `APIFY_KEY_21`), advancing to the next key per query. If a key returns 402/403/429, auto-rotates to the next.

**File:** `apify-proxy.php` (lines 511-549)

### 5 Poll & Dataset Fetch

Frontend polls `?action=check&runId=...` every 3 seconds until `SUCCEEDED`. Then fetches the dataset via `?action=dataset&datasetId=...`. The proxy fetches the FULL Apify dataset (up to 5000 items) to merge into the shared cache.

**File:** `components/LeadSearch.jsx` (polling logic), `apify-proxy.php` (lines 592-706)

## 6 Cache Merge

After fetching live Apify data, `$supabaseCache->mergePlaces()` deduplicates by placeld and upserts new places into `leadscraper_leads_data` with `UserEmail='__cache__'`. Future identical queries are served from this cache for free.

**File:** `lib/supabase_cache.php` (lines 82-108)

## 7 Credit Slicing

`applyCreditSlice()` computes what the user receives:

- Fetches user's already-delivered placelds (free, already paid)
- Checks extras queue (overflow from previous searches)
- Delivers extras FIRST, then new pool leads, capped at 100 per search
- Charges `leadCount * 0.01` credits via Makerkit's `/api/supabase/credits/deduct-leads`
- Records delivered IDs, enqueues overflow, logs the search

**File:** `lib/credits.php` (lines 514-592), `apify-proxy.php` (lines 186-310)

## 8 Lead Scoring

Frontend scores each lead 0-10 based on: unclaimed GBP (+25), no website (+20), no SSL (+10), no reviews (+20), low rating (+10), no social (+5). Leads are tagged as "Hot Lead" (7.5+), "Warm Lead" (5+), "Cool Lead" (2.5+), or "Low Priority".

**File:** `components/LeadSearch.jsx` (lines 40-77), `utils/helpers.js`

## 9 Email Extraction (Background)

For leads with websites, the frontend calls `extract-email.php?url=...` which scrapes the business homepage (then /contact, /about pages) for email addresses. Results are cached in localStorage for 24 hours.

**File:** `extract-email.php` , `utils/emailExtractor.js`

## 10 Save to Supabase

User clicks Save (or auto-bulk-saves). `leads-proxy.php` upserts per-user state rows into `user_leadscraper_leads` with Status='delivered'. Business data stays in the pool table; only user-editable metadata (Notes, LeadScore, ManagerStatus) is stored per-user.

**File:** `leads-proxy.php` (lines 180-258)

## 3. Frontend Architecture (React + Vite)

### Key Frontend Files

| File                        | Role                                                                  |
|-----------------------------|-----------------------------------------------------------------------|
| <code>index.html</code>     | Entry point — Turnstile CAPTCHA gate, OTP login, embed mode detection |
| <code>app.jsx</code>        | Main App component — routing, state management, data persistence      |
| <code>styles.css</code>     | Custom styling                                                        |
| <code>vite.config.js</code> | Vite build config                                                     |

### Component Architecture

| Component                        | Purpose                                                                |
|----------------------------------|------------------------------------------------------------------------|
| <code>Sidebar.jsx</code>         | Navigation sidebar with page links                                     |
| <code>Dashboard.jsx</code>       | Overview stats (leads, emails, reports)                                |
| <code>LeadSearch.jsx</code>      | Core search — keyword/location input, Apify integration, results table |
| <code>LeadDetail.jsx</code>      | Single lead detail view with audit report                              |
| <code>LeadManager.jsx</code>     | Saved leads table — status/notes management, bulk site generation      |
| <code>EmailGenerator.jsx</code>  | AI-powered email generation for leads                                  |
| <code>ReportGenerator.jsx</code> | Lead audit report generation (PDF)                                     |
| <code>ReviewResponder.jsx</code> | Google review response generator                                       |
| <code>PostCreator.jsx</code>     | Social media post creator                                              |
| <code>EmailOutreach.jsx</code>   | Email campaign management                                              |
| <code>Settings.jsx</code>        | User settings                                                          |

---

`SearchableDropdown.jsx`Reusable searchable dropdown for country/state/city

---

## Pages / Routes (client-side)

| Page             | URL param                           | Component                    |
|------------------|-------------------------------------|------------------------------|
| Search           | <code>?page=search</code> (default) | <code>LeadSearch</code>      |
| Dashboard        | <code>?page=dashboard</code>        | <code>Dashboard</code>       |
| Lead Detail      | <code>?page=detail</code>           | <code>LeadDetail</code>      |
| Lead Manager     | <code>?page=leads</code>            | <code>LeadManager</code>     |
| Email Generator  | <code>?page=email-gen</code>        | <code>EmailGenerator</code>  |
| Reports          | <code>?page=reports</code>          | <code>ReportGenerator</code> |
| Review Responder | <code>?page=review</code>           | <code>ReviewResponder</code> |
| Post Creator     | <code>?page=posts</code>            | <code>PostCreator</code>     |
| Email Outreach   | <code>?page=outreach</code>         | <code>EmailOutreach</code>   |
| Settings         | <code>?page=settings</code>         | <code>Settings</code>        |

---

## Utils

| File                                 | Purpose                                                          |
|--------------------------------------|------------------------------------------------------------------|
| <code>utils/emailExtractor.js</code> | Extracts emails from business websites (localStorage cached)     |
| <code>utils/helpers.js</code>        | Lead scoring colors, status badges, date formatting, audit logic |
| <code>utils/indiaData.js</code>      | Country/state/city data for location dropdowns                   |
| <code>utils/map2web.js</code>        | Map2Web site generation client (build → publish → log)           |
| <code>utils/mockData.js</code>       | Mock data for development                                        |
| <code>utils/pdfGenerator.js</code>   | PDF report generation (uses jspdf)                               |

---



## Build Process

```
npm run build
  → vite build (bundles React app to dist/)
  → npm run copy-php (copies .env + all PHP files to dist/)
```

The `copy-php` script copies: `.env`, `send-otp.php`, `apify-proxy.php`, `extract-email.php`, `sheets-proxy.php`, `shorten-url.php`, `leads-proxy.php`, `map2web-proxy.php`, `leadscraper-credits-proxy.php`, `debug.php`, and the `lib/` PHP files.

## 4. PHP Backend Layer

The PHP backend is NOT a framework — it's a collection of standalone proxy files. Each file handles one responsibility:

| File                                       | Purpose                                             | Methods                                               |
|--------------------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| <code>apify-proxy.php</code>               | Apify API proxy with key rotation + cache + billing | run, check, dataset, balance, status, cache           |
| <code>leads-proxy.php</code>               | Lead CRUD (save, load, update, delete, bulk save)   | load, saveLead, bulkSaveLeads, updateLead, deleteLead |
| <code>leadscraper-credits-proxy.php</code> | Credit proxy to Makerkit                            | balance, deduct-leads                                 |
| <code>map2web-proxy.php</code>             | Reverse proxy to Map2Web (app.pixnom.com)           | scrape, deduct, build, publish, log                   |
| <code>extract-email.php</code>             | Website email extraction                            | GET ?url=...                                          |
| <code>send-otp.php</code>                  | Sends OTP via Resend API                            | POST {email, otp}                                     |
| <code>sheets-proxy.php</code>              | Google Apps Script proxy                            | —                                                     |
| <code>shorten-url.php</code>               | URL shortening                                      | —                                                     |
| <code>debug.php</code>                     | Debug endpoint                                      | —                                                     |

### Shared Libraries (lib/)

| File                                | Purpose                                                                         |
|-------------------------------------|---------------------------------------------------------------------------------|
| <code>lib/supabase.php</code>       | Low-level Supabase REST helper (sb_select, sb_insert, sb_update, sb_delete)     |
| <code>lib/supabase_cache.php</code> | SupabaseCache class — query cache, merge, pending serves                        |
| <code>lib/credits.php</code>        | Credit system — balance, deduct, delivered tracking, extras queue, search audit |

## Supabase REST Helper Pattern

```
sb_select(table, query)    → GET  /rest/v1/{table}?{query}
sb_insert(table, rows)     → POST /rest/v1/{table}
sb_update(table, query, p) → PATCH /rest/v1/{table}?{query}
sb_delete(table, query)    → DELETE /rest/v1/{table}?{query}
```

All use PostgREST with the service role key for full access.

## 5. Apify Integration & Key Rotation

### Actor Used

`compass~crawler-google-places` — a free/public Apify actor for scraping Google Maps business listings.

### Key Rotation System

| Feature            | Detail                                                                                                               |
|--------------------|----------------------------------------------------------------------------------------------------------------------|
| Max Keys           | Up to 21 keys ( <code>APIFY_KEY_1</code> through <code>APIFY_KEY_21</code> )                                         |
| Rotation Trigger   | HTTP 402 (quota), 403 (disabled), 429 (rate limit)                                                                   |
| Per-Query Rotation | Each new search advances to next key (round-robin)                                                                   |
| Run Binding        | <code>runKeyMap</code> maps <code>runId</code> → <code>keyIndex</code> so check/dataset use the same key             |
| State Persistence  | <code>.apify_key_state.json</code> file ( <code>currentIndex</code> + <code>runKeyMap</code> + <code>expiry</code> ) |
| Expiry             | <code>runKeyMap</code> entries pruned after 24 hours                                                                 |

### Request Flow

```
Action: run
  → Credit pre-check (Makerkit balance)
  → Cache check (Supabase)
  → If cache HIT: return synthetic runId immediately
  → If cache MISS: POST to Apify actor with current key
  → On success: map runId → keyIndex, advance to next key

Action: check
  → If cached runId: return SUCCEEDED immediately
  → Else: GET actor-runs/{runId} with same key that started it

Action: dataset
  → If cached runId: decode cacheKey from runId, fetch from Supabase cache
  → Apply credit slice, return sliced results
  → If live: GET dataset items, merge into cache, apply credit slice
```

## 6. Credit & Billing System

### Pricing Model

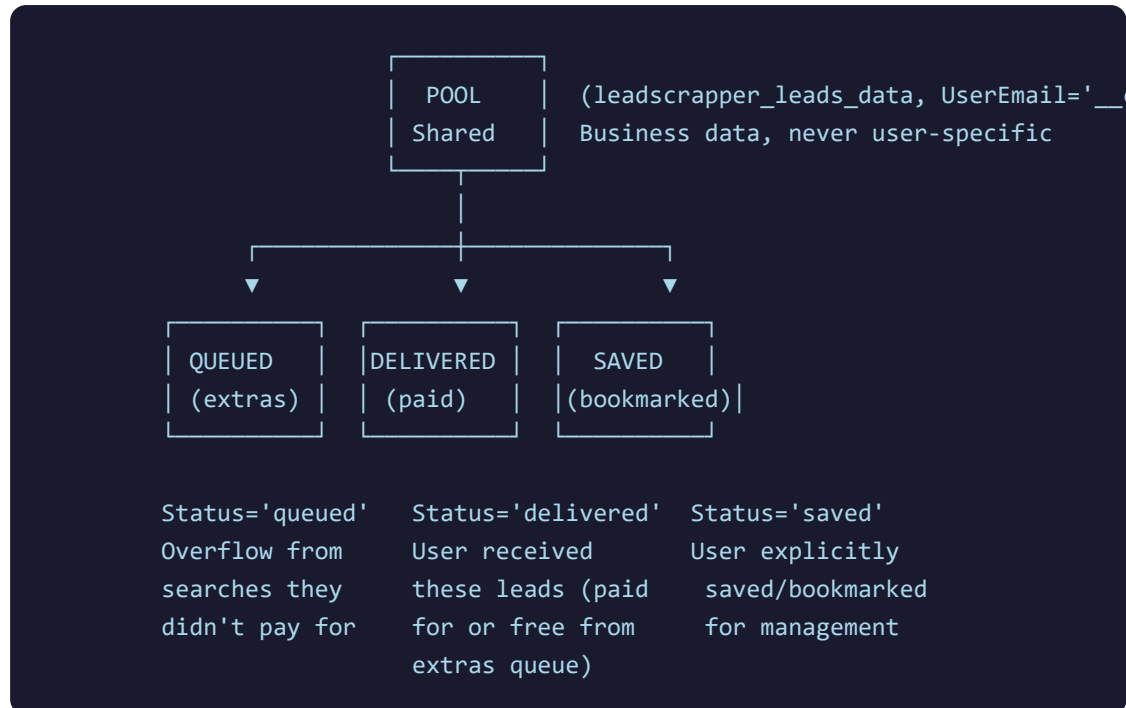
| Unit           | Value                    |
|----------------|--------------------------|
| 1 Credit       | = 100 leads              |
| 1 Lead         | = 0.01 credit            |
| Max per search | 100 leads                |
| Dev emails     | 9999 credits (unlimited) |

### Credit Flow Architecture



```
└─ credits_enqueue_extras() – queue overflow
  └─ credits_log_search() – audit log
```

## Three-Tier Lead State Machine



## Extras Queue Logic

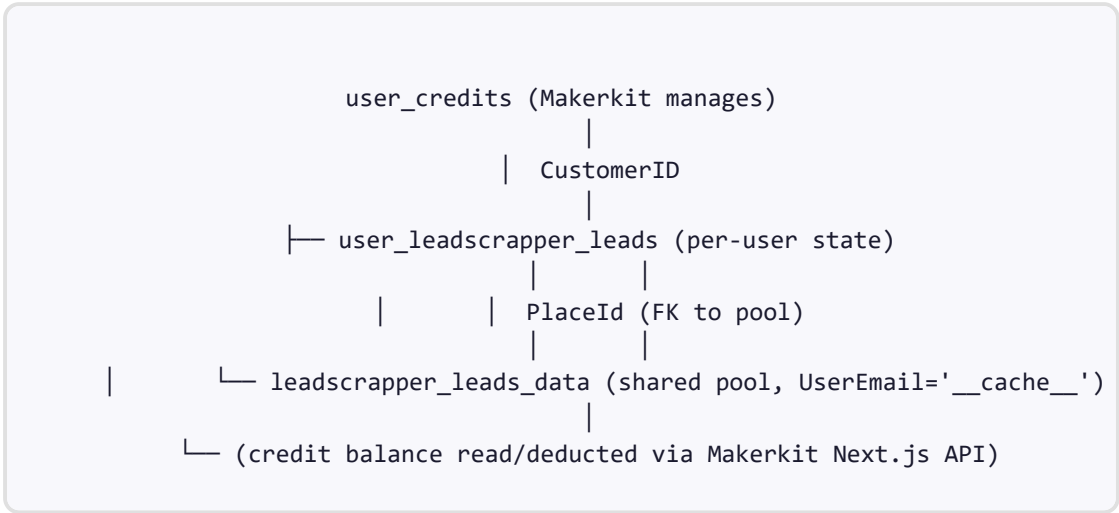
When a search returns more leads than the user's balance allows, the overflow is queued as `Status='queued'` in `user_leadscraper_leads`. On the next search for the same cacheKey, these extras are delivered FIRST (free — already counted). Only truly new leads are charged.

## 7. Supabase Data Layer

### Tables (Post-Migration 003)

| Table                               | Purpose                               | Key Columns                                                                                       |
|-------------------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>user_credits</code>           | Credit wallet (authoritative balance) | Email, Credits (numeric 10,2), CustomerID, UpdatedAt                                              |
| <code>leadscraper_leads_data</code> | Shared business data pool + cache     | UserEmail='__cache__', PlaceId, Title, Address, Phone, Website, SearchString, etc.                |
| <code>user_leadscraper_leads</code> | Per-user lead state                   | CustomerID, UserEmail, PlaceId, SearchString, Status, Notes, LeadScore, ManagerStatus, SearchMeta |

### Table Relationships



### CustomerID Convention

| ID | Email                        | Type |
|----|------------------------------|------|
| 1  | shreyashchandak.lx@gmail.com | Dev  |
| 2  | shriganeshkolhe@gmail.com    | Dev  |

1001+

Real users

Allocated from customer\_id\_seq

## Cache Key Format

```
cacheKey = lowercase(keyword) + '|' + lowercase(location)
```

Examples:

```
"plumber|zip:90001|us"
```

```
"dentist|new york, ny"
```

```
"restaurant|mumbai, india"
```



## 8. Makerkit Integration (Next.js)

LeadScrapper integrates with the Makerkit SaaS boilerplate (Next.js) at `app.pixnom.com` for two purposes: credit management and embed mode.

### Credit API Routes (Deployed to Makerkit)

| Route                                           | Method | Purpose                       | Source File                                             |
|-------------------------------------------------|--------|-------------------------------|---------------------------------------------------------|
| <code>/api/supabase/credits/get</code>          | GET    | Returns user's credit balance | <code>makerkit-api-credits/get/route.ts</code>          |
| <code>/api/supabase/credits/deduct-leads</code> | POST   | Deducts credits for leads     | <code>makerkit-api-credits/deduct-leads/route.ts</code> |

### Makerkit Page Route

`makerkit-leadscraper/page.tsx` → Redirects to `/home/leadscraper/find-leads`

This allows LeadScrapper to be embedded as a page within the Makerkit Next.js app.

### Embed Mode

When loaded inside an iframe in Makerkit ( `?embed=true` ):

- Turnstile CAPTCHA is skipped
- Login page is skipped
- User email is passed via URL param ( `?email=user@example.com` )
- Sidebar is hidden
- First-time users see a minimal email prompt

### Proxy Architecture

```
Browser (on leadscraper.pixnom.com)
|
▼
leadscraper-credits-proxy.php (PHP, same-origin)
| forwards to Makerkit with service token
```

▼  
app.pixnom.com/api/supabase/credits/{get|deduct-leads}  
|  
▼  
Supabase user\_credits table (direct PostgREST access)

## 9. Email Extraction System

### Flow

1. Frontend calls `extract-email.php?url=https://business-website.com`
2. PHP fetches the homepage HTML via cURL
3. Extracts emails from `mailto:` links and general regex matching
4. Filters blacklisted domains (wixpress.com, sentry.io, etc.) and non-person emails (noreply, postmaster)
5. If no emails found on homepage, tries `/contact`, `/contact-us`, `/about`, `/about-us`
6. Returns `{ success, emails: [], source }`

### Client-Side Caching

`utils/emailExtractor.js` caches results in localStorage with a 24-hour TTL. Keyed by domain. Even empty results are cached to avoid re-scraping.

### Blacklisted Domains

wixpress.com, sentry.io, w3.org, schema.org, googleapis.com, google.com, facebook.com, twitter.com, example.com, wordpress.org, jquery.com, cloudflare.com, gstatic.com, gravatar.com, wp.com, squarespace.com, wix.com, weebly.com, godaddy.com

## 10. Map2Web Site Generation Integration

LeadScraper can generate Basic/Gold/Premium websites for any lead via Map2Web.

### Flow

1. User clicks "Generate Sites" on one or more leads in LeadManager
2. `app.jsx` calls `generateSiteForLead(lead)`
3. `utils/map2web.js` converts the lead to a Map2Web business object via `leadToBusiness()`
4. For each tier (1=Basic, 2=Gold, 3=Premium):
  - POST `/map2web-proxy.php?path=build` → returns { filePath, html, repoName }
  - POST `/map2web-proxy.php?path=publish` → pushes to GitHub Pages
5. POST `/map2web-proxy.php?path=log` → records in Map2Web history
6. Tier URLs are persisted back to Supabase ( `Tier1` , `Tier2` , `Tier3` columns)

### Data Mapping: LeadScraper → Map2Web

| LeadScraper Lead               |   | Map2Web Business               |
|--------------------------------|---|--------------------------------|
| <code>business_name</code>     | → | <code>title</code>             |
| <code>address</code>           | → | <code>address</code>           |
| <code>phone</code>             | → | <code>phone</code>             |
| <code>website</code>           | → | <code>website</code>           |
| <code>category</code>          | → | <code>categoryName</code>      |
| <code>rating</code>            | → | <code>totalScore</code>        |
| <code>review_count</code>      | → | <code>reviewsCount</code>      |
| <code>_raw.Title</code>        | → | <code>title (preferred)</code> |
| <code>_raw.Description</code>  | → | <code>description</code>       |
| <code>_raw.OpeningHours</code> | → | <code>openingHours</code>      |

#### Proxy Auth

`map2web-proxy.php` sends `X-Map2Web-Token` header with `MAP2WEB_SERVICE_TOKEN` from `.env`. This authenticates as a trusted server-to-server caller on `app.pixnom.com`.

# 11. Authentication & Security

## Three-Stage Gate

1. **Cloudflare Turnstile** — CAPTCHA gate shown first (skipped on localhost and embed mode)
2. **OTP Login** — email-based 6-digit OTP sent via Resend API, verified client-side
3. **Inactivity Timer** — auto-logout after 10 minutes of no mouse/keyboard activity

## Embed Mode (Makerkit Integration)

When `?embed=true`, the app skips Turnstile + OTP. The email is passed via URL param from the parent Makerkit page. This assumes Makerkit has already authenticated the user.

## Security Considerations

### 1. Client-Side OTP Verification

OTP is generated and verified in the browser. The OTP value is visible in JavaScript. Same trade-off as Map2Web.

### 2. Email in URL Param

In embed mode, the user email is passed as `?email=...` in the URL. This is visible in browser history, server logs, and referrer headers.

### 3. API Keys Hidden

All Apify API keys stay server-side in `.env`. The PHP proxy prevents key exposure to the browser.

## 12. All PHP Proxy Files Reference

| File                                       | Endpoint                                    | Methods  | Description                                   |
|--------------------------------------------|---------------------------------------------|----------|-----------------------------------------------|
| <code>apify-proxy.php</code>               | <code>/apify-proxy.php</code>               | GET/POST | Apify proxy with key rotation, cache, billing |
| <code>leads-proxy.php</code>               | <code>/leads-proxy.php</code>               | GET/POST | Lead CRUD operations                          |
| <code>leadscraper-credits-proxy.php</code> | <code>/leadscraper-credits-proxy.php</code> | GET/POST | Credit balance/deduct proxy to Makerkit       |
| <code>map2web-proxy.php</code>             | <code>/map2web-proxy.php</code>             | POST     | Reverse proxy to Map2Web API                  |
| <code>extract-email.php</code>             | <code>/extract-email.php</code>             | GET      | Website email scraper                         |
| <code>send-otp.php</code>                  | <code>/send-otp.php</code>                  | POST     | OTP email sender via Resend                   |
| <code>sheets-proxy.php</code>              | <code>/sheets-proxy.php</code>              | GET/POST | Google Apps Script proxy                      |
| <code>shorten-url.php</code>               | <code>/shorten-url.php</code>               | POST     | URL shortening service                        |
| <code>debug.php</code>                     | <code>/debug.php</code>                     | GET      | Debug/diagnostic endpoint                     |
| <code>debug-cache.php</code>               | <code>/debug-cache.php</code>               | GET      | Cache inspection endpoint                     |
| <code>clean.php</code>                     | <code>/clean.php</code>                     | GET      | Cleanup utility                               |
| <code>turnstile-key.php</code>             | <code>/turnstile-key.php</code>             | GET      | Turnstile key management                      |

## 13. Database Schema & Migrations

### SQL Migrations

| File                                                      | Purpose                                                  |
|-----------------------------------------------------------|----------------------------------------------------------|
| <code>sql/01_credits_migration.sql</code>                 | Fractional credits + delivered ledger + search audit log |
| <code>sql/02_extras_queue_migration.sql</code>            | Extras queue for overflow leads                          |
| <code>sql/03_deduplicate_leads.sql</code>                 | Deduplication cleanup                                    |
| <code>migrations/001_two_table_redesign.sql</code>        | Two-table architecture (pool + state)                    |
| <code>migrations/001b_add_assign_customer_rpc.sql</code>  | RPC function for CustomerID assignment                   |
| <code>migrations/002_drop_old_tables.sql</code>           | Cleanup old table structure                              |
| <code>migrations/003_consolidate_to_two_tables.sql</code> | Final consolidation to pool + state tables               |
| <code>migrations/20260522000000_credit_bridge.sql</code>  | Credit bridge migration                                  |

### Final Schema (Post-Migration 003)

| Table                               | PK                    | Description                                                                                       |
|-------------------------------------|-----------------------|---------------------------------------------------------------------------------------------------|
| <code>user_credits</code>           | Email                 | Credit wallet. Managed by Makerkit. Columns: Email, Credits (numeric 10,2), CustomerID, UpdatedAt |
| <code>leadscraper_leads_data</code> | (UserEmail, PlacelId) | Shared business data pool. UserEmail='__cache__' for cache rows. ~40 columns of business data.    |

|                                     |                                              |                                                                                                                        |
|-------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>user_leadscraper_leads</code> | (CustomerId, PlacelId, SearchString, Status) | Per-user lead state. Status ∈ {delivered, saved, queued, search}. Carries Notes, LeadScore, ManagerStatus, SearchMeta. |
|-------------------------------------|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|

## Makerkit-Side Tables (app.pixnom.com)

| Table                     | Used By             | Purpose                      |
|---------------------------|---------------------|------------------------------|
| <code>user_credits</code> | Makerkit API routes | Authoritative credit balance |



## 14. Architecture Observations & Gotchas

### 1. PHP Files are Standalone Proxies

No framework, no router, no middleware. Each .php file is its own endpoint. The build script copies them to dist/ alongside the bundled React app.

### 2. Two Supabase Instances

LeadScrapper's Supabase holds leads data + local credit audit tables. Makerkit's Supabase (same or different instance) holds the authoritative `user_credits` table. The PHP layer bridges between them.

### 3. Cache is Permanent

Once a cacheKey has ANY entries in `leadscraper_leads_data`, Apify is NEVER called again for that query. The TTL gate was removed. This saves API costs but means stale data persists.

### 4. Three Lead States

The same lead can exist in three states simultaneously: as a pool row (shared data), as a delivered row (user received it), and as a queued row (overflow from previous search). The system deduplicates carefully.

### 5. Client-Side OTP + Email in URL

Same security trade-offs as Map2Web. OTP verified in browser, email visible in URL params during embed mode.

### 6. Apify Key Rotation is File-Based

`.apify_key_state.json` persists the current key index. On multi-instance deployments, this could cause race conditions. Works fine for single-server setup.

### 7. Embed Mode Bypasses All Auth

When `?embed=true`, the app trusts the email URL param completely. Security depends on the parent Makerkit app properly authenticating users before embedding.

## 8. Map2Web is a Reverse Proxy

The LeadScrapper server proxies all Map2Web API calls through `map2web-proxy.php`. This avoids CORS issues and keeps the service token server-side.

Whitelist: scrape, deduct, build, publish, log.

## 9. Supabase Cache Replaces Firebase

The cache was migrated from Firebase Firestore to Supabase (leadscraper\_leads\_data table). The `SupabaseCache` class is a drop-in replacement with the same interface.

## 10. Makerkit is the Source of Truth for Credits

LeadScrapper never modifies the `user_credits` table directly. All balance reads and deductions go through Makerkit's Next.js API routes, which use the Supabase admin client.